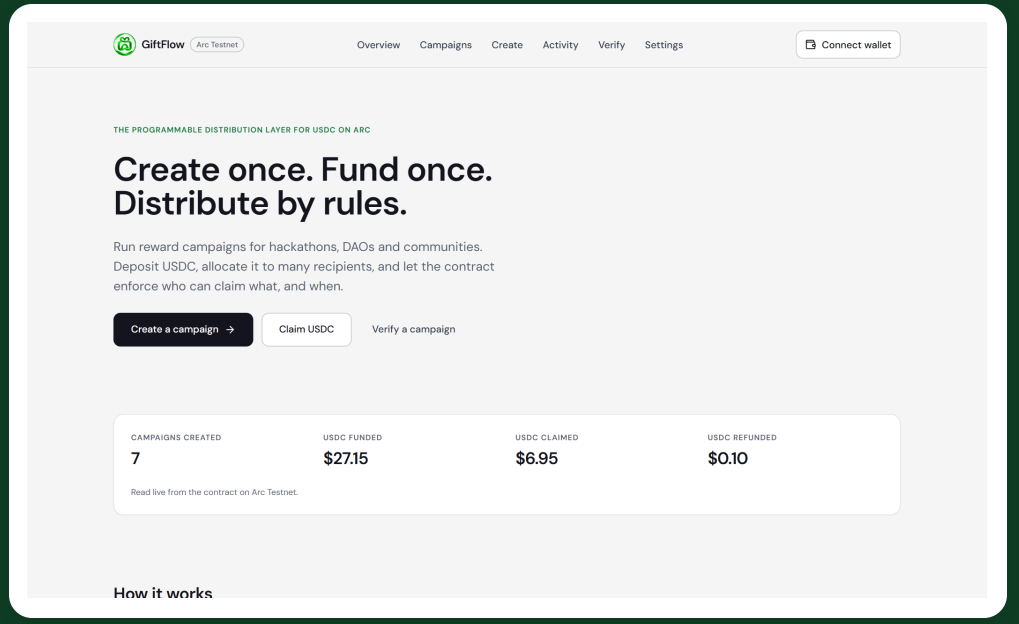
GiftFlow

Programmable USDC distribution on Arc

Create once. Fund once. Distribute by rules.



Live on Arc Testnet

Paying many people in USDC is still manual

One transfer per person

Paying 100 people means 100 transactions, 100 addresses to check and a spreadsheet to reconcile.

Recipients cannot see the money is there

A promise of payment is not a funded payment. Trust is the only guarantee.

Leftovers need chasing

Unclaimed rewards sit in a wallet that someone has to remember to clean up.

50 to 100

recipients in one campaign: the first launch GiftFlow is built to handle.

Create once. Fund once. Distribute by rules

1

Create

Name the campaign, set the claim window and choose how people get paid.

2

Fund

One transaction locks the full USDC total in the contract. Allocations can never exceed it.

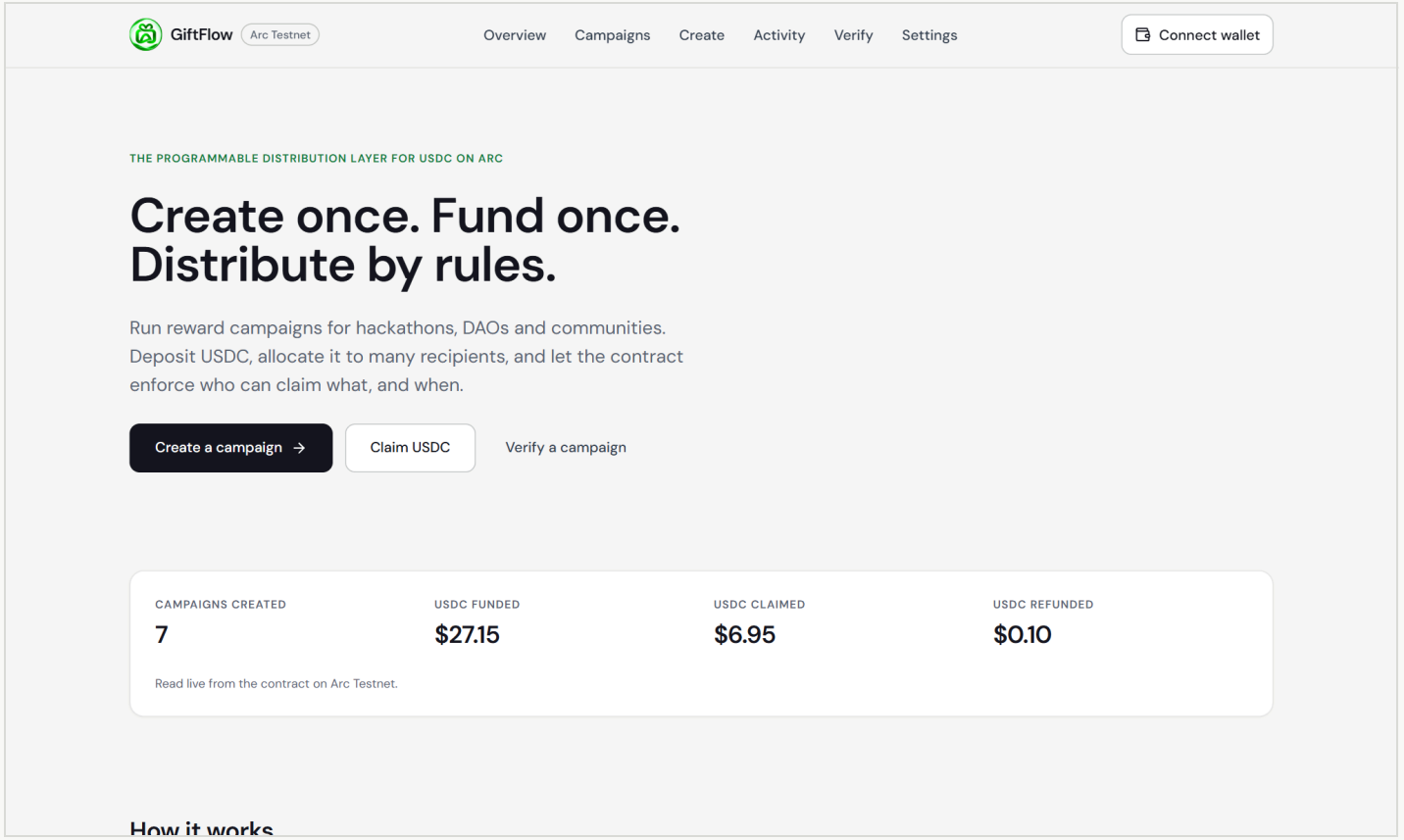
3

Distribute

Recipients claim with a wallet, a link or a code. Unclaimed USDC returns to you after the deadline.

Non-custodial. No database. No backend. Everything that matters lives onchain on Arc.

The product at a glance



Landing page on Arc Testnet

Live totals

Campaigns, USDC funded, claimed and refunded are read straight from the contract.

Three clear actions

Create a campaign, claim USDC, or verify any campaign without a wallet.

No sign-up

Connect an Arc wallet and go. No account, no backend, no stored data.

Three ways to pay people

Wallet list

You know each wallet. Everyone opens one page, connects and claims, or you send to all of them.

Best for salaries and DAO payouts

One shared link

Equal shares with one link. Anyone who has it claims one share until none are left.

Best for open giveaways

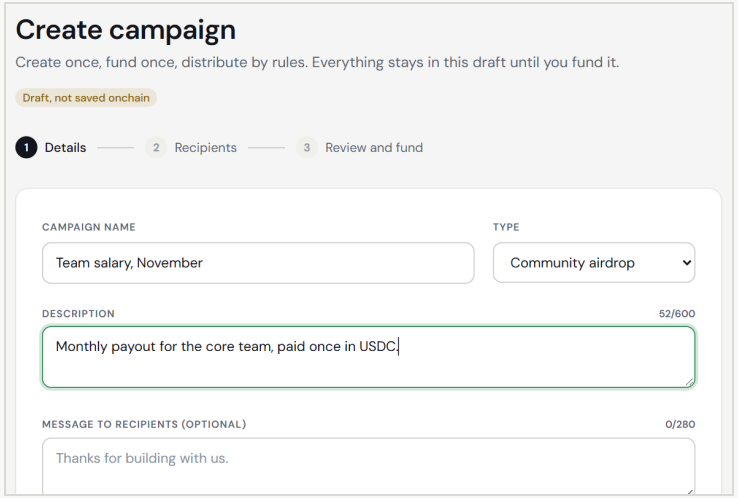
Secret codes

Equal shares with one personal code per person. Each code works once.

Best for exactly one share each

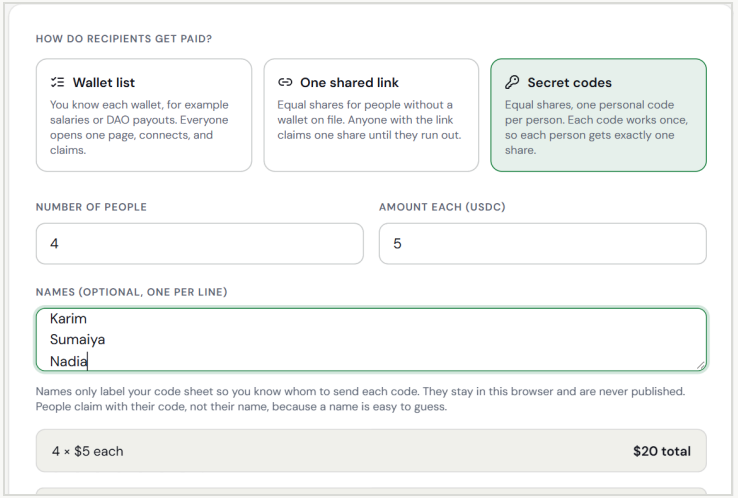
Same contract and same guarantees in every mode. Pick one per campaign.

Create a campaign in three steps



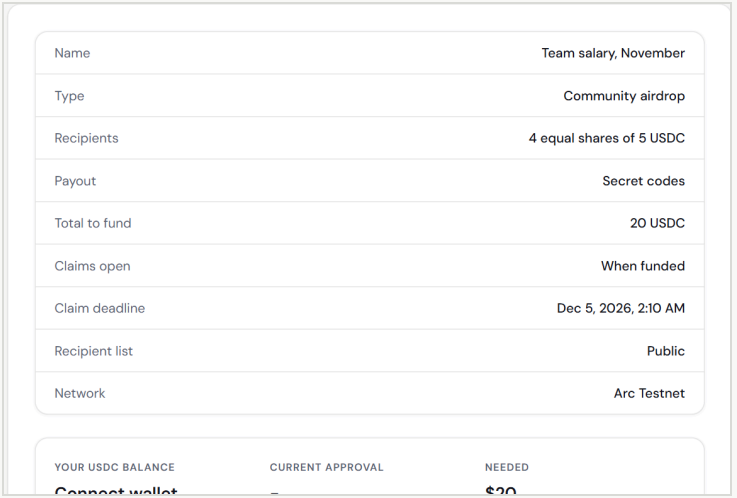
1 Details

Name the campaign, add a message and set the claim window. Choose a public or private recipient list.



2 Recipients

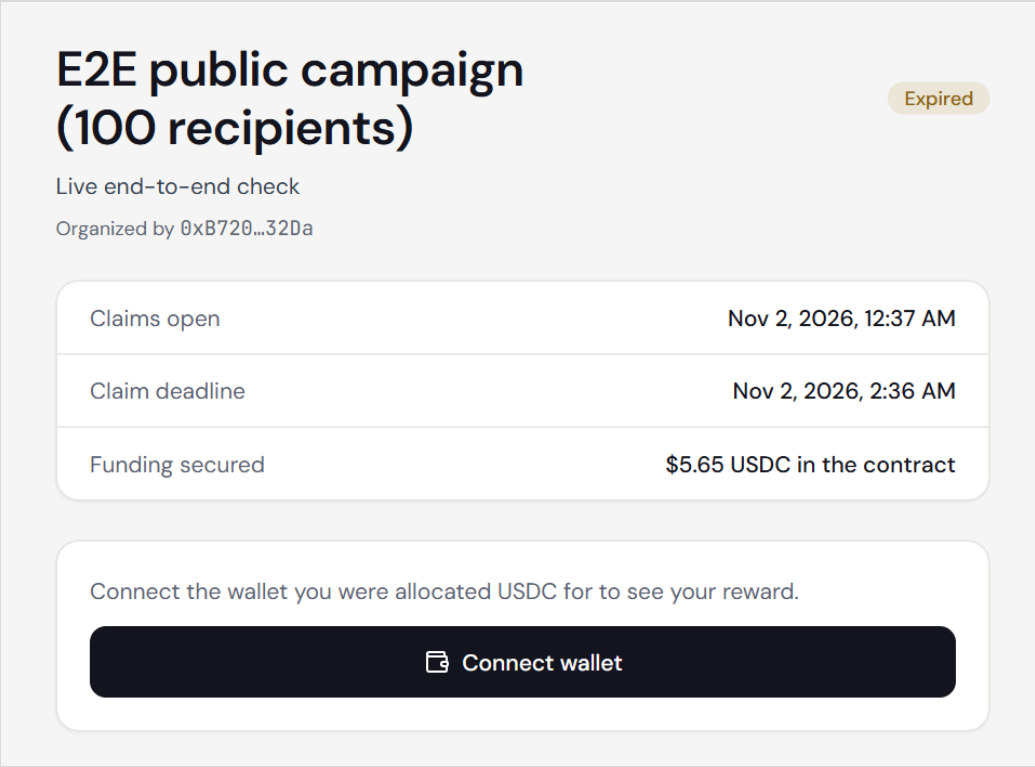
Pick wallet list, one shared link or secret codes. Import a CSV or enter equal shares.



3 Review and fund

Check the total, balance and approval. Fund in one transaction. Success shows only after the chain confirms.

Claiming takes one wallet and one click



Claim page of a live campaign. This one has expired, so claiming is closed.

Connect the wallet

The page shows your reward and the deadline once your wallet is connected.

Eligibility is enforced onchain

The contract checks the proof and the wallet. A hidden button is never the only guard.

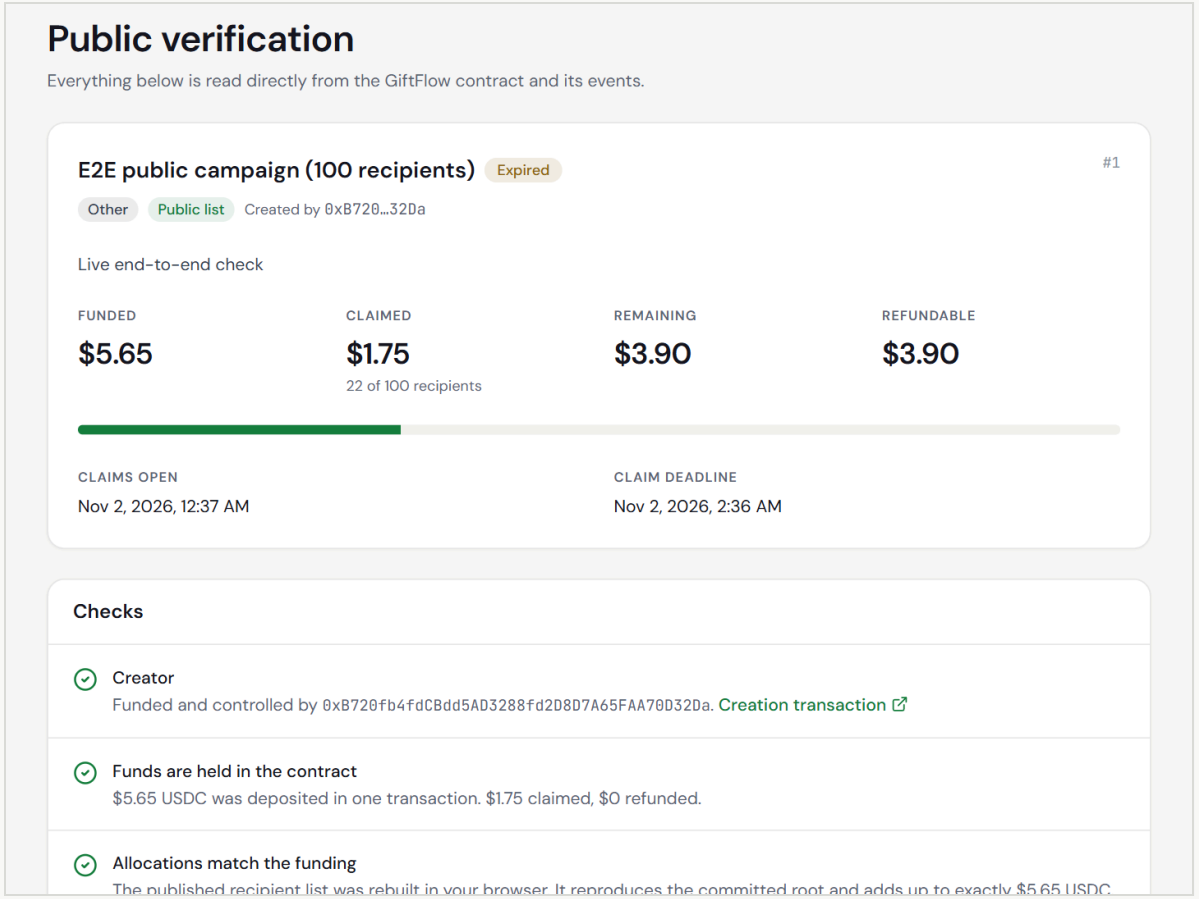
One claim per allocation

Duplicate and replayed claims are rejected by the contract.

USDC goes straight to you

No middleman holds funds. Success shows only after the chain confirms.

Anyone can verify a campaign



Campaign #1: 100 recipients on Arc Testnet

Funded, claimed, remaining

Read live from the contract, with no wallet needed.

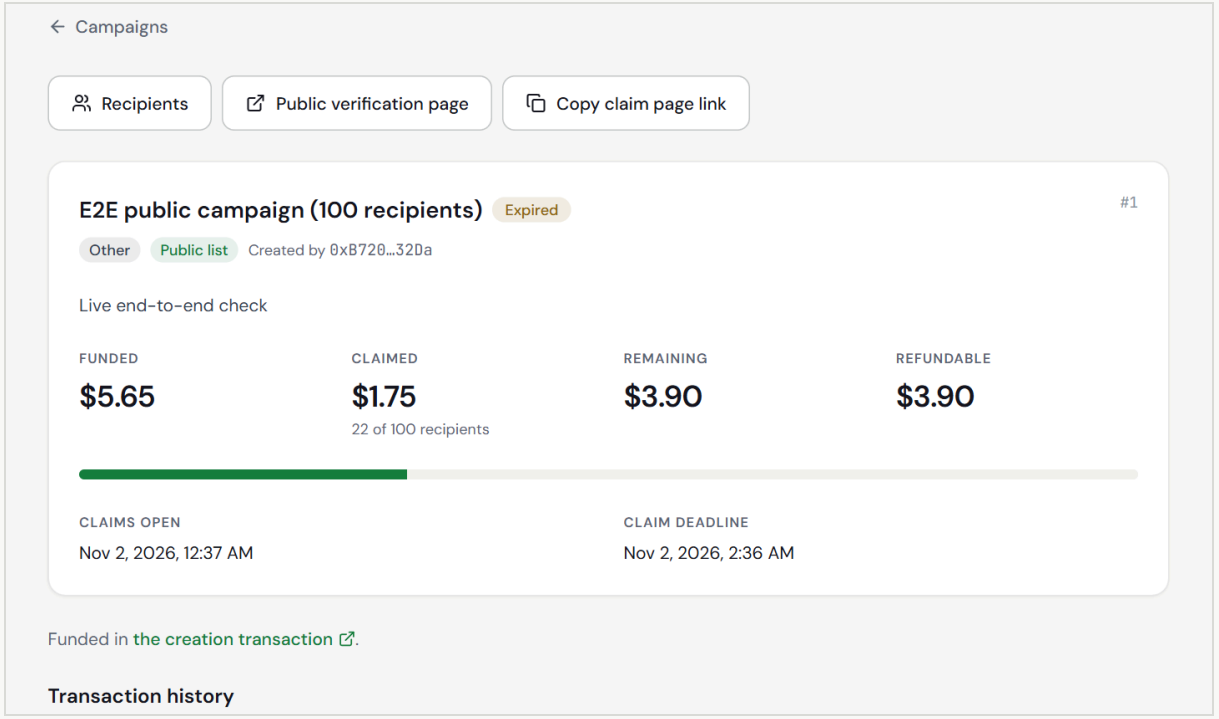
Allocations match the funding

The recipient list is rebuilt in your browser and compared with the onchain commitment.

Every transaction is linked

Creation, claims and refunds point to the Arc block explorer.

Organizers stay in control



Campaign page: funding, claimed, remaining and history

Send directly

Push wallet-list rewards to every wallet, 50 per transaction.

Refund after the deadline

Unclaimed USDC returns to the funding wallet, once.

Cancel before it starts

Scheduled campaigns can be cancelled for a full refund.

Pause never traps funds

Refunds and cancellations still work while paused.

Security is enforced by the contract

Safe transfers

Reentrancy guards, SafeERC20 and state updated before any transfer.

Single-use claims

A bitmap marks every claimed allocation. No duplicates, no replays.

Wallet-bound claims

Signatures name the receiving wallet, so a copied transaction cannot redirect funds.

Funding equals allocation

A Merkle sum tree proves allocations add up to the deposited total.

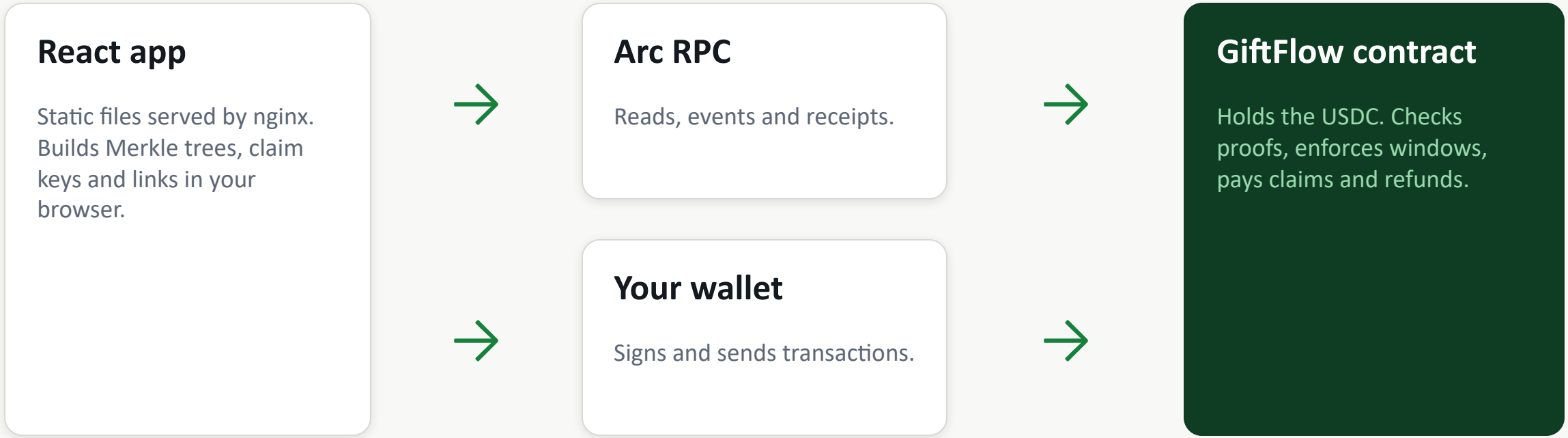
Safe refunds

Creator only, after expiry, once. Works even while paused.

No operator backdoor

The owner can pause and set a cap. It cannot withdraw funds or edit campaigns.

No database, no backend



State lives in contract storage and events. The browser stores three interface preferences only.

Tested before it holds real money

123

Foundry contract tests: unit, fuzz and invariant

7

frontend tests for codes, claim keys and links

100

recipients in live testnet campaign #1

3.5M

gas to create a 1,000-recipient campaign, under the 30M block limit

Proven on Arc Testnet

A live end-to-end script created campaign #1 on the real contract; 22 of its 100 allocations were claimed.

Not yet done

An independent security audit, and any mainnet deployment.

Ready to host and easy to run

Run it anywhere

- Docker image: a Vite build served by nginx
- docker-compose.yml ready for Coolify
- One build variable: the contract address
- Health check at /healthz and security headers

Built with

Solidity	**OpenZeppelin**	**Foundry**
React 18	**Vite**	**TypeScript**
Tailwind CSS	**wagmi**	**viem**

Arc Testnet, chain 5042002, USDC with 6 decimals.

What comes next

Vesting

Release funds over time, with cliffs, per recipient.

Milestones

Pay when a milestone is approved by the organizer.

Attestations

Signed proof of external work unlocks claims.

Independent audit

A third-party review before real value is held.

Indexer

Faster history than public RPC log scans.

Mainnet

Added only after an explicit go-ahead.

A contract cannot see the real world alone, so conditional rewards need an authorized verifier or a signed attestation.



Create once. Fund once. Distribute by rules.

GiftFlow: programmable USDC distribution on Arc

Full documentation is in the docs folder of the repository.